

3D Graphics Engine & Digital Twin Specification

Source: docs/THREE_D_ENGINE.md • Status: Verified & Synchronized

Product: Cubyntra
Organization: Necookie Labs
Status: Implemented & Tested
Module: src/three/ and src/components/cube/

1. Overview & Architectural Goals

The Cubyntra 3D engine provides an interactive, mathematically faithful digital twin of the physical Rubik's Cube. It executes in client-side WebGL via Three.js.

Core Engineering Invariants

- Visual Follows Model: Three.js is strictly a rendering projection. The logical cube state in src/cube/ is the single source of truth.
- Zero Floating-Point Accumulation Drift: Repeated matrix multiplications during continuous face turns inevitably degrade floating-point orthogonality without rigorous snapping. All layer rotations round cubie coordinates to integer space $\{-1, 0, 1\}$ upon animation completion.
- Smooth 60 FPS Interpolation: Smooth easing transitions between move states.
- Context Loss Resilience: Graceful handling of WebGL device loss and dynamic canvas resize.

2. 27-Cubie Spatial Decomposition

A standard 3 x 3 x 3 Rubik's cube consists of 26 physical pieces plus 1 core mechanism, indexed in discrete spatial coordinates:

$x, y, z \in \{-1, 0, 1\}$

Layers along axes:
X-Axis: Left ($x = -1$), Middle ($x = 0$), Right ($x = +1$)
Y-Axis: Down ($y = -1$), Middle ($y = 0$), Up ($y = +1$)
Z-Axis: Back ($z = -1$), Middle ($z = 0$), Front ($z = +1$)

Cubie Classification:

- Corners (8 pieces): $|x| + |y| + |z| = 3$. Have 3 colored faces, 3 internal faces.
- Edges (12 pieces): $|x| + |y| + |z| = 2$. Have 2 colored faces, 4 internal faces.
- Centers (6 pieces): $|x| + |y| + |z| = 1$. Have 1 colored face, 5 internal faces.
- Core (1 piece): $|x| + |y| + |z| = 0$. Hidden internal pivot mechanism.

Speedcube PBR Mesh Construction & Shared Buffer Geometries

Rather than rudimentary BoxGeometry cubes, Cubyntra constructs cubies using precision-extruded competition speedcube components:

- Chamfered ABS Body: Extruded rounded shape with bevel (`createRoundedBoxGeometry(0.985, 0.11, 5)`) creating smooth aerodynamic edges. Shaded with matte black ABS plastic `MeshPhysicalMaterial` (`#15161b`, roughness 0.55, clearcoat 0.25).
- 3D Beveled Facelet Tiles: Tactile 3D tiles (`createRoundedTileGeometry(0.84, 0.13, 0.018)`) with quadratic corner fillets and bevel. Mounted onto external faces with slight normal offset, eliminating z-fighting and mimicking physical speedcube tiles.

- PBR Clearcoat Materials: High-gloss specular coating with MeshPhysicalMaterial (roughness 0.32, clearcoat 0.70, clearcoatRoughness 0.18, envMapIntensity 0.90) and authentic competition speedcube color palette:
- White (#f4f5f0), Yellow (#ffd500), Green (#00b35c), Blue (#0a5c9f), Red (#e8132e), Orange (#ff7a00).
- Buffer Geometry Sharing: Shared module-level geometry singletons (getSharedBodyGeometry and getSharedTileGeometry) reuse vertex and index buffers across all 27 cubies, guaranteeing minimal GPU memory overhead and zero garbage collection spikes.

3. Dynamic Layer Rotation & Zero-Drift Snapping

3.1 Grouping and Detaching Mechanics

Directly rotating individual cubies along world axes creates non-affine shearing when rotations are compounded.

Cubyntra solves this via temporary dynamic grouping:

- Selection: Identify the 9 cubies whose current rounded world coordinate matches the target slice:
- For U: all cubies where $\text{round}(y) = +1$.
- For R: all cubies where $\text{round}(x) = +1$.
- For F: all cubies where $\text{round}(z) = +1$.
- Parenting: A pivot THREE.Group is a child of the cube group, reset to identity before each turn. Each of the 9 cubies is moved into it with `pivotGroup.attach(cubie)`. Because the pivot shares the cube group's frame, the turning layer keeps the cube's orbit rotation and idle bob. (A pivot parented to the scene made the layer detach and turn about a world axis whenever the view was orbited.)
- Interpolated Turn: The pivot's rotation is advanced inside the single render loop, never a separate `requestAnimationFrame` chain, using the frame's clamped delta (at most 100 ms) and a cubic ease-in-out:

$\theta(t) = \theta_{\text{target}} \cdot \text{ease}(t / T)$

Half turns take 1.5x as long as quarter turns.

- Detaching: At θ_{target} each cubie is moved back with `rootCubeGroup.attach(cubie)`.
- Orthonormal Snapping:
- Position is rounded to the integer grid.
- Orientation is snapped by rounding the rotation matrix's first two basis vectors to the nearest signed axes and deriving the third by cross product (`snapQuaternionToCubeGroup`). The result is always one of the 24 exact cube orientations with determinant +1. Rounding Euler angles per component, the previous method, is not a valid snap near gimbal lock.
- Repaint by facing: After each turn, `syncWithCubeState` repaints tiles from the logical state, choosing each tile by the direction it points now (`CubieMesh.setColorFacing`), not by the side it was built on.

3.2 Move Queue & Playback

- `animateMove(move, targetState, durationMs)` queues the turn and resolves when that turn has been snapped. Turns are never dropped, however fast they are requested.
- `requestSync(state)` repaints immediately when idle. While turns are queued it stores the state and applies it once the queue drains, so colors never jump to a post-move state mid-turn.
- `SolveControls` plays a solution as an async loop: advance the store, await the turn, rest, repeat. At 1x a turn takes 300 ms plus a 250 ms rest, and both scale with speed. Next and Previous animate too; Previous plays the inverse turn.

Verified in tests/`three-motion.test.ts` by driving the real engine frame by frame and reading the state back from cubie geometry and from material colors.

4. Directional 3D Move Arrows

To guide users through solving moves physically, Cubyntra renders animated 3D directional arrows projecting above the active rotating face:



- Geometry: Parametric arc curve (`THREE.QuadraticBezierCurve3`) extruded with `THREE.TubeGeometry`, terminating in a conical arrow tip (`THREE.ConeGeometry`).
- Dynamic Placement: Projected at a normal offset of +0.8 units from the center of the active face.
- Rotation Sense: The arrow arc curve automatically flips based on clockwise (U), counter-clockwise (U'), or double-turn (U2) move specifications.

• Pulsing Animation: Subtle emissive shader pulse synchronizing with playback tempo.

5. Camera & Orbit

- Perspective camera: 40° vertical field of view, looking at the origin. Dragging rotates the cube group (pitch clamped to $\pm 60^\circ$) the scroll wheel zooms.
- Fitted distance: By default the camera sits where the cube's bounding sphere (radius $1.5\sqrt{3}$) fits inside the narrower of the vertical and horizontal view angles:
$$d = \frac{1.5\sqrt{3}}{\sin(\min(\theta_v, \theta_h)/2)}$$

The cube is therefore never clipped, from any angle or mid-turn, in wide, square or tall containers. The distance follows container resizes until the user zooms; Reset View restores it.

- Damping: Orbit and zoom ease with $k = 1 - e^{-7.67 \Delta t}$, which matches the original 0.12 per frame at 60 Hz and behaves the same at any refresh rate.
- Clock: THREE.Timer, deliberately not tied to the Page Visibility API, so turns still complete where frames keep firing while document.hidden is true. The per-frame delta clamp prevents fast-forwarding when a tab returns.

5.1 Studio Environment & Contact Shadow

- PMREM Studio Reflections: Real-time environment reflections generated via THREE.PMREMGGenerator using synthetic studio light panels (overhead softbox, left/right rim strips, and front fill), casting natural glossy highlights across beveled tile edges.
- Contact Shadow Plane: Projected radial gradient texture positioned at $y = -2.35$ with dynamic scale and opacity breathing in sync with subtle floating idle bobbing.
- Color Science & Tone Mapping: Configured with THREE.SRGBColorSpace and THREE.ACESFilmicToneMapping (exposure 1.05) for realistic physical rendering.

6. Rendering Performance & Resource Lifecycle

- Disposal: Thorough resource cleanup hooks disposing geometries, materials, and textures when the component unmounts to prevent memory leaks in Single Page Applications.
- Render Loop: Renders every animation frame; the browser pauses it in background tabs.